16.1 Discussion of K Nearest Neighbour

ভূমিকা

Machine Learning-এ Classification এবং Regression সমস্যার সমাধানের জন্য ব্যবহৃত সবচেয়ে সহজ এবং জনপ্রিয় অ্যালগরিদমগুলোর মধ্যে একটি হলো **K-Nearest Neighbour (KNN)**।
KNN একটি **Supervised Learning Algorithm**, অর্থাৎ এটি প্রশিক্ষণের সময় Labelled Data ব্যবহার করে। নতুন কোনো ডেটা আসলে KNN সেই ডেটার সবচেয়ে কাছাকাছি থাকা Training Data-এর সাহায্যে Prediction করে।
KNN-এর সবচেয়ে বড় সুবিধা হলো এটি বোঝা এবং বাস্তবায়ন করা খুবই সহজ। অন্য অনেক অ্যালগরিদমের মতো এখানে জটিল Mathematical Model তৈরি করতে হয় না।

KNN কী?

K-Nearest Neighbour এমন একটি অ্যালগরিদম যা নতুন একটি Data Point-এর Class নির্ধারণ করার জন্য তার আশেপাশের K সংখ্যক নিকটবর্তী প্রতিবেশী (Neighbour) খুঁজে বের করে।
এখানে,

K = কতজন Neighbour বিবেচনা করা হবে
Nearest = সবচেয়ে কাছাকাছি
Neighbour = Training Dataset-এর Data Points

নতুন ডেটা যে ক্লাসের প্রতিবেশীদের মধ্যে বেশি সংখ্যক সদস্য পাবে, সেটিকেই Prediction হিসেবে নির্বাচন করা হবে।

বাস্তব জীবনের উদাহরণ

ধরুন আপনি একটি নতুন ফল পেয়েছেন এবং জানতে চান এটি আপেল নাকি কমলা।
আপনার কাছে আগে থেকেই কিছু ফলের তথ্য আছে:

ওজন	রং	ফল
150g	লাল	আপেল

170g	লাল	আপেল
140g	সবুজ	আপেল
180g	কমলা	কমলা
200g	কমলা	কমলা

এখন একটি নতুন ফলের ওজন 175g।

KNN এই ফলটির কাছাকাছি থাকা ফলগুলো খুঁজবে এবং তাদের Label দেখে সিদ্ধান্ত নেবে ফলটি আপেল নাকি কমলা।

KNN কীভাবে কাজ করে?

KNN মূলত নিম্নলিখিত ধাপগুলো অনুসরণ করে:

ধাপ ১: K এর মান নির্বাচন করা

প্রথমে কতজন প্রতিবেশী বিবেচনা করা হবে তা নির্ধারণ করা হয়।

উদাহরণ:

K = 3

অর্থাৎ সবচেয়ে কাছাকাছি ৩টি Data Point দেখা হবে।

ধাপ ২: Distance গণনা করা

নতুন Data Point এবং Training Dataset-এর প্রতিটি Data Point-এর মধ্যে দূরত্ব নির্ণয় করা হয়।

সবচেয়ে বেশি ব্যবহৃত Distance হলো Euclidean Distance।

ধাপ ৩: K সংখ্যক Neighbour নির্বাচন করা

যেসব Data Point সবচেয়ে কম দূরত্বে অবস্থান করছে তাদের নির্বাচন করা হয়।

ধাপ ৪: Majority Voting

নির্বাচিত K সংখ্যক Neighbour-এর মধ্যে কোন Class সবচেয়ে বেশি আছে তা গণনা করা হয়।

যে Class-এর সংখ্যা বেশি হবে সেটিকেই Final Prediction ধরা হবে।

Distance কেন গুরুত্বপূর্ণ?

KNN-এর মূল ভিত্তিই হলো Distance।

কারণ KNN ধরে নেয়:

"যেসব Data Point একে অপরের কাছাকাছি থাকে তারা সাধারণত একই ধরনের হয়।"

এই ধারণাকে বলা হয়:

Similarity Principle

অর্থাৎ কাছাকাছি অবস্থিত ডেটাগুলো একে অপরের সাথে বেশি সাদৃশ্যপূর্ণ।

KNN-এ Distance এর ভূমিকা

ধরুন দুইজন শিক্ষার্থীর তথ্য রয়েছে:

Height	Weight
170	65
172	68

দুইজনের Height এবং Weight প্রায় একই।

তাই তাদের Distance কম হবে।

ফলে KNN তাদের একই Class-এ রাখার সম্ভাবনা বেশি।

KNN Classification বনাম Regression

KNN শুধু Classification নয়, Regression সমস্যাও সমাধান করতে পারে।

Classification

Output হয় কোনো Category।

উদাহরণ:

Spam
Not Spam

অথবা

Cat
Dog

Regression

Output হয় Numerical Value।

উদাহরণ:

- House Price
- Temperature
- Sales Prediction

Classification-এ Majority Voting ব্যবহার করা হয়।

Regression-এ Neighbour-দের Average ব্যবহার করা হয়।

KNN-এর বৈশিষ্ট্য

KNN-এর কয়েকটি গুরুত্বপূর্ণ বৈশিষ্ট্য হলো:

Non-Parametric Algorithm

এটি কোনো নির্দিষ্ট Mathematical Equation ধরে নেয় না।

Lazy Learning Algorithm

Training Phase-এ KNN প্রকৃতপক্ষে কোনো Model তৈরি করে না।

Training Data-ই সংরক্ষণ করে রাখে।

Prediction-এর সময় গণনা করে।

Instance-Based Learning

এটি Training Examples-এর উপর ভিত্তি করে সিদ্ধান্ত নেয়।

নতুন ডেটার জন্য কাছাকাছি উদাহরণগুলো ব্যবহার করা হয়।

KNN-এর সুবিধা

১. সহজে বোঝা যায়

Machine Learning-এর সবচেয়ে সহজ অ্যালগরিদমগুলোর একটি।

২. Training খুব দ্রুত

কারণ Model Training বলতে বিশেষ কিছু নেই।

৩. Multi-Class Classification করতে পারে

একাধিক Class নিয়ে কাজ করতে সক্ষম।

৪. Non-Linear Data Handle করতে পারে

অনেক ক্ষেত্রে জটিল Boundary শিখতে পারে।

KNN-এর অসুবিধা

১. Prediction ধীর হতে পারে

কারণ প্রতিবার Prediction-এর সময় পুরো Dataset-এর সাথে Distance গণনা করতে হয়।

২. Large Dataset-এ ব্যয়বহুল

Dataset বড় হলে Computation Cost বেড়ে যায়।

৩. Noise-এর প্রতি সংবেদনশীল

ভুল বা Outlier Data থাকলে Prediction খারাপ হতে পারে।

৪. Feature Scaling প্রয়োজন

একটি Feature-এর মান অনেক বড় হলে Distance Calculation-এ সেটি অতিরিক্ত প্রভাব ফেলতে পারে।

তাই সাধারণত Standardization বা Normalization ব্যবহার করা হয়।

KNN-এর ব্যবহার ক্ষেত্র

KNN বিভিন্ন ক্ষেত্রে ব্যবহৃত হয়:

- Image Classification
- Handwriting Recognition
- Recommendation System
- Medical Diagnosis
- Fraud Detection
- Customer Segmentation
- Pattern Recognition

সারসংক্ষেপ

K-Nearest Neighbour (KNN) একটি সহজ কিন্তু শক্তিশালী Supervised Learning Algorithm। এটি নতুন Data Point-এর Class নির্ধারণ করার জন্য সবচেয়ে কাছাকাছি থাকা K সংখ্যক Neighbour ব্যবহার করে। KNN-এর মূল ভিত্তি হলো Distance Measurement এবং Similarity Principle। এটি বিশেষভাবে Classification সমস্যায় জনপ্রিয় হলেও Regression সমস্যাতেও ব্যবহার করা যায়। যদিও বড় Dataset-এর ক্ষেত্রে Prediction ধীর হতে পারে, তবুও এর সরলতা ও কার্যকারিতার কারণে এটি Machine Learning-এর অন্যতম গুরুত্বপূর্ণ অ্যালগরিদম হিসেবে বিবেচিত হয়।

16.2 Multiclass Classification with KNN

ভূমিকা

পূর্ববর্তী মডিউলে আমরা K-Nearest Neighbour (KNN) অ্যালগরিদমের মৌলিক ধারণা সম্পর্কে জেনেছি। সেখানে মূলত Binary Classification-এর উদাহরণ ব্যবহার করা হয়েছিল, যেখানে একটি ডেটা শুধুমাত্র দুইটি ক্লাসের যেকোনো একটিতে শ্রেণিবদ্ধ করা হয়।

কিন্তু বাস্তব জীবনের অধিকাংশ সমস্যায় দুইটির বেশি ক্লাস থাকে। যেমন:

- একটি ফুল Iris-Setosa, Iris-Versicolor অথবা Iris-Virginica হতে পারে।
- একটি প্রাণীর ছবি Cat, Dog অথবা Rabbit হতে পারে।
- একটি ইমেইল Spam, Promotion অথবা Personal হতে পারে।

এ ধরনের সমস্যাকে বলা হয় **Multiclass Classification**।

KNN অ্যালগরিদম খুব স্বাভাবিকভাবেই Multiclass Classification পরিচালনা করতে পারে। এজন্য অতিরিক্ত কোনো বিশেষ পরিবর্তনের প্রয়োজন হয় না।

Multiclass Classification কী?

যখন Target Variable-এ দুইটির বেশি Category বা Class থাকে, তখন সেই Classification সমস্যাকে Multiclass Classification বলা হয়।

উদাহরণ:

ধরুন একটি ফুলের Dataset আছে।

সম্ভাব্য ক্লাস:

- Setosa
- Versicolor
- Virginica

এখানে মোট ৩টি Class রয়েছে।

সুতরাং এটি একটি Multiclass Classification Problem।

KNN কীভাবে Multiclass Classification করে?

KNN-এর মূল ধারণা একই থাকে।

ধাপগুলো হলো:

- নতুন Data Point-এর Distance গণনা করা।
- সবচেয়ে কাছের K সংখ্যক Neighbour নির্বাচন করা।
- Neighbour-দের Class গণনা করা।
- যে Class সবচেয়ে বেশি বার পাওয়া যায় সেটিকে Prediction হিসেবে নির্বাচন করা।

এটিকে Majority Voting বলা হয়।

উদাহরণ

ধরুন আমাদের Dataset-এ তিনটি Class আছে:

- Red
- Blue
- Green

নতুন একটি Data Point-এর জন্য K = 5 নির্বাচন করা হয়েছে।

নিকটবর্তী ৫টি Neighbour-এর Class:

Red

Blue

Red

Green

Red

গণনা করলে:

Red = 3

Blue = 1

Green = 1

সবচেয়ে বেশি Red পাওয়া গেছে।

সুতরাং Prediction হবে:

Red

আরেকটি উদাহরণ

ধরুন K = 7

নিকটবর্তী Neighbour-গুলো হলো:

Blue

Blue

Green

Green

Green

Red

Blue

গণনা করলে:

Blue = 3

Green = 3

Red = 1

এখানে Tie তৈরি হয়েছে।

অর্থাৎ Blue এবং Green সমান সংখ্যক পাওয়া গেছে।

এ ধরনের ক্ষেত্রে বিভিন্ন Implementation বিভিন্ন কৌশল ব্যবহার করে:

- ছোট Distance-কে অগ্রাধিকার দেওয়া
- K পরিবর্তন করা
- Weighted Voting ব্যবহার করা

Weighted Voting সম্পর্কে আমরা পরবর্তী মডিউলে বিস্তারিত আলোচনা করব।

Majority Voting ধারণা

Multiclass KNN-এর সবচেয়ে গুরুত্বপূর্ণ অংশ হলো Majority Voting।

ধরুন:

K = 9

Neighbour-দের Class:

Class A = 4

Class B = 3

Class C = 2

তাহলে Prediction হবে:

Class A

কারণ এটি সবচেয়ে বেশি সংখ্যক Neighbour-এর Class।

কেন **KNN Multiclass Problem-**এ ভালো কাজ করে?

KNN-এর জন্য Binary কিংবা Multiclass Classification-এর মধ্যে বিশেষ কোনো পার্থক্য নেই।

কারণ KNN কোনো Mathematical Equation শেখে না।

বরং এটি সরাসরি Neighbour-এর Class দেখে সিদ্ধান্ত নেয়।

তাই নতুন Class যোগ হলেও অ্যালগরিদমের কাঠামো পরিবর্তন করতে হয় না।

Iris Dataset: Multiclass Classification-এর সবচেয়ে জনপ্রিয় উদাহরণ

Machine Learning-এর সবচেয়ে বিখ্যাত Datasetগুলোর একটি হলো Iris Dataset।

এখানে ৩টি ফুলের প্রজাতি রয়েছে:

- Iris Setosa
- Iris Versicolor
- Iris Virginica

প্রতিটি ফুলের জন্য ৪টি Feature রয়েছে:

- Sepal Length
- Sepal Width
- Petal Length
- Petal Width

Target Variable:

- Setosa
- Versicolor
- Virginica

সুতরাং এটি একটি Multiclass Classification Problem।

Iris Dataset-এ KNN-এর ধারণাগত কাজ

ধরুন নতুন একটি ফুল এসেছে যার Feature হলো:

Sepal Length = 5.9

Sepal Width = 3.0

Petal Length = 4.2

Petal Width = 1.5

KNN প্রথমে Training Dataset-এর প্রতিটি ফুলের সাথে Distance বের করবে।

এরপর K সংখ্যক সবচেয়ে কাছের ফুল নির্বাচন করবে।

ধরা যাক K = 5

নিকটবর্তী ফুলগুলোর Class:

Versicolor

Versicolor

Virginica

Versicolor

Versicolor

এখানে:

Versicolor = 4

Virginica = 1

Prediction:

Versicolor

Scikit-Learn ব্যবহার করে Multiclass KNN

নিচের উদাহরণে Iris Dataset ব্যবহার করে Multiclass Classification করা হয়েছে।

C/C++

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score

# Load Dataset
iris = load_iris()

X = iris.data
y = iris.target

# Train Test Split
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
```

```
    test_size=0.2,
    random_state=42
)

# Model
knn = KNeighborsClassifier(n_neighbors=5)

# Training
knn.fit(X_train, y_train)

# Prediction
y_pred = knn.predict(X_test)

# Accuracy
accuracy = accuracy_score(y_test, y_pred)

print("Accuracy:", accuracy)
```

ব্যাখ্যা

প্রথমে Iris Dataset Load করা হয়েছে।

```
C/C++

1iris = load_iris()
```

এরপর Feature এবং Target আলাদা করা হয়েছে।

```
C/C++

1X = iris.data

2y = iris.target
```

তারপর Training এবং Testing Dataset তৈরি করা হয়েছে।

```
C/C++

1train_test_split()
```

KNN Model তৈরি করা হয়েছে।

```
C/C++

1KNeighborsClassifier(n_neighbors=5)
```

এখানে K = 5 নেওয়া হয়েছে।
মডেল Training করা হয়েছে।

```
C/C++

1knn.fit(X_train, y_train)
```

Prediction করা হয়েছে।

```
C/C++

1y_pred = knn.predict(X_test)
```

Accuracy বের করা হয়েছে।

C/C++

```
1accuracy_score(y_test, y_pred)
```

Predicted Class Name বের করা

KNN সাধারণত Class Number Return করে।

Iris Dataset-এ:

0 = Setosa

1 = Versicolor

2 = Virginica

Class Name দেখতে:

C/C++

```
1print(iris.target_names)
```

Output:

C/C++

```
1['setosa' 'versicolor' 'virginica']
```

নতুন ফুল Predict করা

ধরুন একটি নতুন ফুলের তথ্য:

C/C++

```
1new_flower = [[5.9, 3.0, 4.2, 1.5]]
```

Prediction:

```
C/C++

1prediction = knn.predict(new_flower)

2
3print(prediction)
```

Output:

```
C/C++

1[1]
```

Class Name বের করতে:

```
C/C++

1print(iris.target_names[prediction[0]])
```

Output:

```
C/C++

1versicolor
```

Multiclass Classification-এ KNN-এর সুবিধা

সহজ বাস্তবায়ন

অতিরিক্ত জটিলতা ছাড়াই একাধিক Class Handle করতে পারে।

নতুন Class যোগ করা সহজ

Dataset-এ নতুন Category যোগ করলেও অ্যালগরিদমের কাঠামো পরিবর্তন করতে হয় না।

Non-Linear Decision Boundary তৈরি করতে পারে

অনেক জটিল Pattern শিখতে সক্ষম।

ছোট Dataset-এ কার্যকর

কম Data থাকলেও ভালো ফলাফল দিতে পারে।

Multiclass Classification-এ KNN-এর সীমাবদ্ধতা

Prediction ধীর হতে পারে

প্রতিটি Prediction-এর সময় Distance Calculation করতে হয়।

Large Dataset-এ Cost বেশি

Dataset বড় হলে Computation Time বৃদ্ধি পায়।

K-এর মানের উপর নির্ভরশীল

ভুল K নির্বাচন করলে Accuracy কমে যেতে পারে।

Feature Scaling প্রয়োজন

Feature-এর Scale ভিন্ন হলে Distance Calculation ভুলভাবে প্রভাবিত হতে পারে।

সারসংক্ষেপ

Multiclass Classification হলো এমন Classification Problem যেখানে দুইটির বেশি Class থাকে। KNN অ্যালগরিদম স্বাভাবিকভাবেই Multiclass Classification পরিচালনা করতে পারে। এটি নতুন Data Point-এর সবচেয়ে কাছের K সংখ্যক Neighbour খুঁজে বের করে এবং Majority Voting-এর মাধ্যমে Class নির্ধারণ করে। Iris Dataset হলো Multiclass KNN শেখার সবচেয়ে জনপ্রিয় উদাহরণ। Scikit-Learn-এর KNeighborsClassifier ব্যবহার করে খুব সহজেই Multiclass Classification Model তৈরি, Training এবং Prediction করা যায়।

16.4 Choosing the Distance Metrics and Choosing the K Value

ভূমিকা

K-Nearest Neighbour (KNN) অ্যালগরিদমের কার্যকারিতা মূলত দুটি গুরুত্বপূর্ণ বিষয়ের উপর নির্ভর করে:

- কোন Distance Metric ব্যবহার করা হচ্ছে
- K-এর মান কত নির্বাচন করা হচ্ছে

একই Dataset-এ ভিন্ন Distance Metric অথবা ভিন্ন K Value ব্যবহার করলে সম্পূর্ণ ভিন্ন ফলাফল পাওয়া যেতে পারে। তাই KNN Model তৈরির সময় Distance Metric এবং K Value নির্বাচন করা অত্যন্ত গুরুত্বপূর্ণ।

এই মডিউলে আমরা বিভিন্ন Distance Metric, তাদের ব্যবহার, এবং K-এর উপযুক্ত মান নির্বাচন করার কৌশল সম্পর্কে আলোচনা করব।

Distance Metric কী?

KNN-এর মূল ধারণা হলো নতুন Data Point-এর সবচেয়ে কাছের Neighbour খুঁজে বের করা।

কিন্তু "কাছের" বলতে কী বোঝায়?

এটি নির্ধারণ করার জন্য Distance Metric ব্যবহার করা হয়।

Distance Metric হলো এমন একটি পদ্ধতি যা দুইটি Data Point-এর মধ্যে দূরত্ব পরিমাপ করে।

কেন Distance Metric গুরুত্বপূর্ণ?

ধরুন দুইজন শিক্ষার্থীর তথ্য আছে:

Student A:

- Height = 170
- Weight = 65

Student B:

- Height = 172
- Weight = 68

Student C:

- Height = 190
- Weight = 95

স্বাভাবিকভাবেই Student A এবং Student B একে অপরের কাছাকাছি।

Distance Metric ব্যবহার করে KNN এই সাদৃশ্য পরিমাপ করে।

সবচেয়ে জনপ্রিয় **Distance Metrics**

Scikit-Learn-এর KNN-এ একাধিক Distance Metric ব্যবহার করা যায়।

সবচেয়ে প্রচলিত Distance Metrics হলো:

- Euclidean Distance
- Manhattan Distance
- Minkowski Distance

Euclidean Distance

এটি সবচেয়ে বেশি ব্যবহৃত Distance Metric।

এটি দুইটি Point-এর সরলরেখার দূরত্ব (Straight Line Distance) পরিমাপ করে।

দুইটি Point:

(x1, y1)

(x2, y2)

এর Euclidean Distance:

d = sqrt((x2 - x1)^2 + (y2 - y1)^2)

Euclidean Distance উদাহরণ

ধরি:

Point A = (2, 3)

Point B = (5, 7)

Distance:

d = sqrt((5 - 2)^2 + (7 - 3)^2)

d = sqrt(9 + 16)

d = sqrt(25)

d = 5

সুতরাং Distance = 5

Euclidean Distance-এর বৈশিষ্ট্য

সুবিধা

- সবচেয়ে জনপ্রিয়
- Continuous Data-এর জন্য কার্যকর
- বোঝা সহজ

অসুবিধা

- Feature Scale ভিন্ন হলে সমস্যা হয়
- Outlier-এর প্রভাব বেশি

Manhattan Distance

Manhattan Distance-কে City Block Distance-ও বলা হয়।

এখানে সরলরেখার দূরত্ব নয়, বরং Grid ধরে চলার দূরত্ব গণনা করা হয়।

Formula:

$d = |x2 - x1| + |y2 - y1|$

Manhattan Distance উদাহরণ

Point A = (2, 3)

Point B = (5, 7)

Distance:

$d = |5 - 2| + |7 - 3|$

$d = 3 + 4$

$d = 7$

সুতরাং Distance = 7

Manhattan Distance-এর বৈশিষ্ট্য

সুবিধা

- Outlier-এর প্রভাব তুলনামূলক কম

- High Dimensional Data-এ ভালো কাজ করতে পারে

অসুবিধা

- সবসময় বাস্তব দূরত্বকে সঠিকভাবে উপস্থাপন করে না

Minkowski Distance

Minkowski Distance হলো Euclidean এবং Manhattan Distance-এর Generalized Form।

Formula:

d = (Σ|xi - yi|^p)^(1/p)

এখানে,

p = 1 হলে Manhattan Distance

p = 2 হলে Euclidean Distance

Scikit-Learn-এ Distance Metric নির্বাচন

Euclidean Distance:

```C/C++
from sklearn.neighbors import KNeighborsClassifier

knn = KNeighborsClassifier(
    n_neighbors=5,
    metric='euclidean'
)
```

Manhattan Distance:

C/C++

```
from sklearn.neighbors import KNeighborsClassifier

knn = KNeighborsClassifier(
    n_neighbors=5,
    metric='manhattan'
)
```

Minkowski Distance:

C/C++

```
from sklearn.neighbors import KNeighborsClassifier

knn = KNeighborsClassifier(
    n_neighbors=5,
    metric='minkowski'
)
```

Distance Metric পরিবর্তন করলে কী হয়?

একই Dataset-এর ক্ষেত্রে:

Euclidean Distance এক ধরনের Neighbour নির্বাচন করতে পারে।
Manhattan Distance অন্য ধরনের Neighbour নির্বাচন করতে পারে।

ফলে Prediction পরিবর্তিত হতে পারে।

তাই Distance Metric-কে একটি Hyperparameter হিসেবে বিবেচনা করা হয়।

K Value কী?

KNN-এর K নির্দেশ করে কতজন Neighbour ব্যবহার করা হবে।

উদাহরণ:

K = 3

মানে সবচেয়ে কাছের ৩টি Data Point ব্যবহার করা হবে।

K-এর প্রভাব বোঝা

ধরি একটি নতুন Data Point এসেছে।

তার নিকটবর্তী Neighbour-দের Class:

Red

Red

Blue

K = 1 হলে

শুধু সবচেয়ে কাছের Neighbour দেখা হবে।

Prediction:

Red

K = 3 হলে

Red = 2

Blue = 1

Prediction:

Red

K খুব ছোট হলে কী হয়?

ধরি:

K = 1

তাহলে Model খুব বেশি Sensitive হয়ে যায়।

Training Data-এর Noise বা Outlier-এর কারণে ভুল Prediction হতে পারে।
এটিকে বলা হয়:
Overfitting

K খুব বড় হলে কী হয়?

ধরি Dataset-এ:
Class A = 50
Class B = 10
Class C = 10
এখন যদি:
K = 50
হয়ে যায়, তাহলে অধিকাংশ ক্ষেত্রেই Class A জিতে যাবে।
ফলে Model নতুন Pattern ধরতে পারবে না।
এটিকে বলা হয়:
Underfitting

K নির্বাচন করার সাধারণ নিয়ম

একটি জনপ্রিয় Thumb Rule হলো:
K = sqrt(N)
এখানে,
N = Training Sample সংখ্যা
উদাহরণ:
যদি Dataset-এ 100 Sample থাকে
তাহলে:
K = sqrt(100)
K = 10
যদি Dataset-এ 400 Sample থাকে
তাহলে:
K = sqrt(400)
K = 20

Odd K কেন ব্যবহার করা হয়?

Binary Classification-এ সাধারণত Odd Value ব্যবহার করা হয়।

যেমন:

K = 3
K = 5
K = 7

কারণ এতে Tie হওয়ার সম্ভাবনা কমে যায়।

বিভিন্ন K Value পরীক্ষা করা

নিচের কোডে একাধিক K ব্যবহার করে Accuracy পরীক্ষা করা হয়েছে।

C/C++

```
1from sklearn.datasets import load_iris

2from sklearn.model_selection import train_test_split

3from sklearn.neighbors import KNeighborsClassifier

4from sklearn.metrics import accuracy_score

5

6iris = load_iris()

7

8X = iris.data

9y = iris.target
```

```
10
11X_train, X_test, y_train, y_test = train_test_split(
12    X,
13    y,
14    test_size=0.2,
15    random_state=42
16)
17
18for k in range(1, 16):
19
20    knn = KNeighborsClassifier(
21        n_neighbors=k
22    )
23
24    knn.fit(X_train, y_train)
25
26    y_pred = knn.predict(X_test)
27
28    accuracy = accuracy_score(y_test, y_pred)
```

```
29
30    print(f"K = {k}, Accuracy = {accuracy:.4f}")
```

Best K খুঁজে বের করা

ধরি ফলাফল হলো:

C/C++

```
1K = 1  Accuracy = 0.9667
2K = 2  Accuracy = 0.9667
3K = 3  Accuracy = 1.0000
4K = 4  Accuracy = 1.0000
5K = 5  Accuracy = 1.0000
6K = 6  Accuracy = 1.0000
7K = 7  Accuracy = 0.9667
```

এখানে K = 3, 4, 5, 6 সবচেয়ে ভালো ফল দিয়েছে।

সাধারণত ছোট K-এর মধ্যে সবচেয়ে স্থিতিশীল মান নির্বাচন করা হয়।

Distance Metric এবং K একসাথে পরীক্ষা করা

বাস্তবে আমরা সাধারণত একাধিক Metric এবং K Value পরীক্ষা করি।

উদাহরণ:

C/C++

```
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score

metrics = ['euclidean', 'manhattan']

for metric in metrics:

    for k in range(1, 11):

        knn = KNeighborsClassifier(
            n_neighbors=k,
            metric=metric
        )

        knn.fit(X_train, y_train)

        y_pred = knn.predict(X_test)

        accuracy = accuracy_score(y_test, y_pred)
```

```
20
21        print(metric, k, accuracy)
```

এভাবে কোন Combination সবচেয়ে ভালো কাজ করছে তা দেখা যায়।

Feature Scaling-এর গুরুত্ব

Distance-based Algorithm হওয়ায় KNN-এ Feature Scaling অত্যন্ত গুরুত্বপূর্ণ।

ধরি:

Age = 20 থেকে 60

Income = 20,000 থেকে 500,000

এখানে Income Feature Distance Calculation-এ আধিপত্য করবে।

তাই KNN ব্যবহার করার আগে সাধারণত StandardScaler বা MinMaxScaler ব্যবহার করা হয়।

উদাহরণ:

C/C++

```
1from sklearn.preprocessing import StandardScaler
2
3scaler = StandardScaler()
4
5X_scaled = scaler.fit_transform(X)
```

Distance Metric এবং K নির্বাচন করার বাস্তব নির্দেশনা

Euclidean Distance দিয়ে শুরু করুন

এটি অধিকাংশ সমস্যায় ভালো কাজ করে।

Feature Scaling করুন
Scaling ছাড়া KNN-এর Performance কমে যেতে পারে।

একাধিক K পরীক্ষা করুন
শুধু K = 5 ব্যবহার না করে বিভিন্ন K পরীক্ষা করুন।

Validation Score দেখুন
শুধু Training Accuracy নয়, Validation বা Test Accuracy বিবেচনা করুন।

Grid Search ব্যবহার করুন
বড় প্রজেক্টে একাধিক Metric এবং K Value পরীক্ষা করার জন্য Grid Search ব্যবহার করা হয়।

সারসংক্ষেপ
KNN-এর Performance মূলত দুটি বিষয়ের উপর নির্ভর করে: Distance Metric এবং K Value। Distance Metric নির্ধারণ করে কোন Data Point-গুলো Neighbour হিসেবে বিবেচিত হবে, আর K নির্ধারণ করে কতজন Neighbour ব্যবহার করা হবে। Euclidean, Manhattan এবং Minkowski হলো সবচেয়ে প্রচলিত Distance Metrics। K খুব ছোট হলে Overfitting এবং খুব বড় হলে Underfitting হতে পারে। তাই বিভিন্ন K এবং Distance Metric পরীক্ষা করে সবচেয়ে ভালো Combination নির্বাচন করা প্রয়োজন। Feature Scaling-ও KNN-এর একটি অত্যন্ত গুরুত্বপূর্ণ ধাপ, কারণ Distance Calculation সরাসরি Feature-এর Scale-এর উপর নির্ভর করে।

16.6 Implementation of KNN using Scikit Learn

ভূমিকা
পূর্ববর্তী মডিউলগুলোতে আমরা K-Nearest Neighbour (KNN) অ্যালগরিদমের তাত্ত্বিক ধারণা, Multiclass Classification, Distance Metrics এবং K Value নির্বাচন সম্পর্কে আলোচনা করেছি।
এই মডিউলে আমরা Scikit-Learn ব্যবহার করে একটি সম্পূর্ণ KNN Classification Pipeline তৈরি করব। এখানে Wine Dataset ব্যবহার করে Data Loading, Data Splitting, Feature Scaling, Model Training এবং Model Evaluation-এর পুরো প্রক্রিয়া ধাপে ধাপে দেখানো হবে।

Wine Dataset পরিচিতি
এই উদাহরণে আমরা Scikit-Learn-এর Built-in Wine Dataset ব্যবহার করব।
Wine Dataset-এ বিভিন্ন ধরনের Wine-এর রাসায়নিক বৈশিষ্ট্য (Chemical Properties) দেওয়া আছে।
Dataset-এ মোট ১৩টি Feature রয়েছে এবং Target Variable-এ ৩টি Class রয়েছে।
অর্থাৎ এটি একটি Multiclass Classification Problem।

প্রয়োজনীয় Library Import

প্রথমে প্রয়োজনীয় Library Import করতে হবে।

C/C++

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline

from sklearn.datasets import load_wine

from sklearn.neighbors import KNeighborsClassifier

from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score
)

from sklearn.metrics import (
    confusion_matrix,
    classification_report
)
```

Dataset Load করা

Scikit-Learn-এর Built-in Wine Dataset Load করা হবে।

```
C/C++


1df = load_wine(as_frame=True).frame
```

Dataset-এর কয়েকটি Row দেখা যাক।

```
C/C++


1df.sample(5)
```

Output:

```
C/C++


1       alcohol  malic_acid  ash  ...  proline  target
212       13.75        1.73  ...     1150       0
395       12.47        1.52  ...      880       1
4142      13.52        3.17  ...      520       2
566       13.11        1.01  ...      680       1
6158      14.34        1.68  ...      780       2
```

Dataset-এর Shape দেখা

Dataset-এ কত Row এবং Column আছে তা দেখার জন্য:

```
C/C++


1df.shape
```

Output:

```
C/C++


1(178, 14)
```

এখানে:

178 টি Sample
14 টি Column

Dataset Information দেখা

C/C++

```
1df.info()
```

Output:

C/C++

```
1<class 'pandas.core.frame.DataFrame'>
2RangeIndex: 178 entries
3Data columns (total 14 columns)
```

এখান থেকে বোঝা যায়:

Missing Value নেই
সব Feature Numeric

Statistical Summary দেখা

C/C++

```
1df.describe()
```

এটি Dataset-এর Mean, Standard Deviation, Minimum, Maximum ইত্যাদি তথ্য দেখাবে।

Target Variable Distribution দেখা

Target Class গুলোর Distribution দেখতে:

```
C/C++

1sns.countplot(data=df, x='target')
2plt.show()
```

এই Plot থেকে Class Distribution সম্পর্কে ধারণা পাওয়া যায়।

Feature এবং Target আলাদা করা

Machine Learning Model-এ Input Feature এবং Output Target আলাদা করতে হয়।

```
C/C++

1X = df.drop(['target'], axis=1)
2
3y = df['target']
```

এখানে:

X = Input Features

y = Target Variable

Train-Test Split

Dataset-কে Training এবং Testing অংশে ভাগ করা হবে।

```
C/C++

1X_train, X_test, y_train, y_test = train_test_split(
```

```
    X,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y
)
```

Train-Test Split-এর Parameter ব্যাখ্যা

test_size=0.2

মোট Dataset-এর ২০% Testing Data হিসেবে ব্যবহৃত হবে।

random_state=42

প্রতিবার একই Result পাওয়ার জন্য ব্যবহার করা হয়।

stratify=y

প্রতিটি Class-এর অনুপাত Training এবং Testing Dataset-এ একই রাখে।

Training Data দেখা

C/C++

```
X_train
```

এখানে Training Feature Dataset দেখা যাবে।

কয়টি Class আছে দেখা

C/C++

```
1y_train.nunique()
```

Output:

C/C++

```
13
```

অর্থাৎ Dataset-এ ৩টি Class রয়েছে।

KNN-এর আগে Feature Scaling কেন প্রয়োজন?

KNN Distance-based Algorithm।

Distance Calculation-এর সময় Feature-এর Scale গুরুত্বপূর্ণ।

ধরুন:

Feature A:

0 থেকে 10

Feature B:

0 থেকে 10000

এক্ষেত্রে Feature B Distance Calculation-এ আধিপত্য করবে।

তাই Feature Scaling করা প্রয়োজন।

Pipeline ব্যবহার করে KNN Model তৈরি

Scikit-Learn-এর Pipeline ব্যবহার করে Scaling এবং Model Training একসাথে করা যায়।

C/C++

```
1knn_model = Pipeline(
```

```
    steps=[
        ('scaler', StandardScaler()),
        ('model', KNeighborsClassifier(
            n_neighbors=15,
            metric='minkowski',
            p=2
        ))
    ]
)
```

Pipeline-এর ব্যাখ্যা

এখানে Pipeline-এর দুটি ধাপ রয়েছে।

ধাপ ১: StandardScaler

```
C/C++

('scaler', StandardScaler())
```

এটি Feature Scaling করবে।

ধাপ ২: KNN Model

```
C/C++

('model', KNeighborsClassifier())
```

এটি Classification Model।

KNN Hyperparameter ব্যাখ্যা

C/C++

```
1KNeighborsClassifier(
2    n_neighbors=15,
3    metric='minkowski',
4    p=2
5)
```

n_neighbors=15

K = 15

অর্থাৎ Prediction-এর সময় ১৫টি Neighbour বিবেচনা করা হবে।

metric='minkowski'

Distance Metric হিসেবে Minkowski Distance ব্যবহার করা হচ্ছে।

p=2

Minkowski Distance-এ:

p = 1 হলে Manhattan Distance

p = 2 হলে Euclidean Distance

অর্থাৎ এখানে Euclidean Distance ব্যবহার করা হচ্ছে।

Model Training

Training Dataset ব্যবহার করে Model Fit করা হবে।

C/C++

```
1knn_model.fit(X_train, y_train)
```

এই ধাপে:

- Feature Scaling শেখা হবে
- Training Data সংরক্ষণ করা হবে
- KNN Prediction-এর জন্য প্রস্তুত হবে

Prediction করা

Testing Data-এর জন্য Prediction করা হবে।

```
C/C++


1y_pred = knn_model.predict(X_test)
2
3y_pred
```

Output:

```
C/C++


1array([0, 1, 2, 0, 1, ...])
```

এগুলো Predicted Class।

Accuracy Calculation

Model কত শতাংশ সঠিক Prediction করেছে তা বের করার জন্য Accuracy ব্যবহার করা হয়।

Formula:

Accuracy = Correct Predictions / Total Predictions

Scikit-Learn ব্যবহার করে:

```
C/C++


1accuracy = accuracy_score(
2    y_test,
3    y_pred
4)
5
6print(accuracy)
```

Output:

C/C++

```
10.97
```

অর্থাৎ Model প্রায় 97% Accuracy অর্জন করেছে।

সম্পূর্ণ কোড

C/C++

```
1import numpy as np
2import pandas as pd
3import matplotlib.pyplot as plt
4import seaborn as sns
5
6from sklearn.model_selection import train_test_split
7from sklearn.preprocessing import StandardScaler
8from sklearn.pipeline import Pipeline
9
10from sklearn.datasets import load_wine
11
12from sklearn.neighbors import KNeighborsClassifier
13
14from sklearn.metrics import (
15    accuracy_score,
16    precision_score,
17    recall_score,
18    confusion_matrix,
19    classification_report
20)
```

```

df = load_wine(as_frame=True).frame

X = df.drop(['target'], axis=1)
y = df['target']

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y
)

knn_model = Pipeline(
    steps=[
        ('scaler', StandardScaler()),
        ('model', KNeighborsClassifier(
            n_neighbors=15,
            metric='minkowski',
            p=2
        ))
    ]
)

knn_model.fit(X_train, y_train)

y_pred = knn_model.predict(X_test)

accuracy = accuracy_score(y_test, y_pred)
```

সারসংক্ষেপ

এই মডিউলে আমরা Scikit-Learn ব্যবহার করে একটি সম্পূর্ণ KNN Classification Pipeline তৈরি করেছি। Wine Dataset Load করার পর Feature এবং Target আলাদা করা হয়েছে, Train-Test Split করা হয়েছে, StandardScaler দিয়ে Feature Scaling করা হয়েছে এবং KNeighborsClassifier ব্যবহার করে Model Training করা হয়েছে। এরপর Accuracy, Precision, Recall, Confusion Matrix এবং Classification Report ব্যবহার করে Model-এর Performance মূল্যায়ন করা হয়েছে। বাস্তব Machine Learning Project-এ KNN Implementation-এর এটি একটি পূর্ণাঙ্গ Workflow।

16.7 Confusion Matrix and Classification Report

ভূমিকা

পূর্ববর্তী মডিউলে আমরা KNN Model তৈরি করেছি এবং Accuracy Score ব্যবহার করে Model-এর Performance মূল্যায়ন করেছি।

কিন্তু শুধুমাত্র Accuracy সবসময় একটি Model-এর প্রকৃত Performance বোঝানোর জন্য যথেষ্ট নয়।

ধরুন একটি Model-এর Accuracy 95%।

এটি দেখতে খুব ভালো মনে হলেও, Model হয়তো কোনো একটি Class-এর Prediction বারবার ভুল করছে। শুধুমাত্র Accuracy দেখে আমরা এই সমস্যাটি বুঝতে পারব না।

এই কারণে Machine Learning-এ Accuracy-এর পাশাপাশি আরও কিছু গুরুত্বপূর্ণ Evaluation Metric ব্যবহার করা হয়, যেমন:

- Confusion Matrix
- Precision
- Recall
- F1-Score
- Classification Report

এই মডিউলে আমরা এগুলোর ধারণা এবং Scikit-Learn ব্যবহার করে Implementation শিখব।

Model Evaluation কেন প্রয়োজন?

Machine Learning Model তৈরি করার মূল উদ্দেশ্য হলো নতুন Data-এর জন্য সঠিক Prediction করা।

Model Training শেষ হওয়ার পরে আমাদের জানতে হবে:

- কতগুলো Prediction সঠিক হয়েছে?
- কতগুলো Prediction ভুল হয়েছে?
- কোন Class সবচেয়ে বেশি ভুল Predict হয়েছে?
- কোন Class সবচেয়ে ভালো Predict হয়েছে?

এই প্রশ্নগুলোর উত্তর পাওয়ার জন্য Evaluation Metrics ব্যবহার করা হয়।

Accuracy Score

Accuracy হলো সবচেয়ে সহজ Evaluation Metric।

Formula:

Accuracy = Correct Predictions / Total Predictions

ধরুন:

মোট Prediction = 100

সঠিক Prediction = 95

তাহলে,

Accuracy = 95 / 100

Accuracy = 0.95

অর্থাৎ

Accuracy = 95%

Accuracy বের করা

C/C++

```
1accuracy = accuracy_score(
2    y_test,
3    y_pred
4)
5
6print(accuracy)
```

উদাহরণ Output:

C/C++

```
10.9722
```

অর্থাৎ Model প্রায় 97.22% Prediction সঠিক করেছে।

Accuracy-এর সীমাবদ্ধতা

Accuracy সবসময় Model-এর প্রকৃত Performance প্রকাশ করে না।

ধরুন:

Dataset-এ

- Class A = 95%
- Class B = 5%

যদি Model সবসময় Class A Predict করে,

তাহলে Accuracy হবে:

95%

কিন্তু Model Class B কখনোই সঠিকভাবে Predict করতে পারবে না।

এ কারণেই Confusion Matrix এবং Classification Report ব্যবহার করা হয়।

Confusion Matrix কী?

Confusion Matrix হলো একটি Table যা দেখায়:

- কতগুলো Prediction সঠিক হয়েছে
- কতগুলো Prediction ভুল হয়েছে
- কোন Class-এর সাথে কোন Class Confuse হয়েছে

এটি Classification Model Evaluation-এর সবচেয়ে গুরুত্বপূর্ণ Toolগুলোর একটি।

Binary Classification-এর Confusion Matrix

ধরুন একটি Email Spam Detection Model আছে।

দুটি Class:

- Spam
- Not Spam

Confusion Matrix-এর চারটি অংশ থাকে:

True Positive (TP)
True Negative (TN)
False Positive (FP)
False Negative (FN)

True Positive (TP)

Actual Positive

এবং

Predicted Positive

উভয়ই Positive।

উদাহরণ:

Spam Email-কে Spam Predict করা।

True Negative (TN)

Actual Negative

এবং

Predicted Negative

উভয়ই Negative।

উদাহরণ:

Normal Email-কে Normal Predict করা।

False Positive (FP)

Actual Negative

কিন্তু Predicted Positive।

উদাহরণ:

Normal Email-কে Spam Predict করা।

এটিকে Type-I Error বলা হয়।

False Negative (FN)

Actual Positive

কিন্তু Predicted Negative।

উদাহরণ:

Spam Email-কে Normal Predict করা।

এটিকে Type-II Error বলা হয়।

Multiclass Classification-এ Confusion Matrix

আমাদের Wine Dataset-এ ৩টি Class রয়েছে।

- Class 0
- Class 1
- Class 2

সেক্ষেত্রে Confusion Matrix হবে 3 × 3।

প্রতিটি Row নির্দেশ করে:

Actual Class

প্রতিটি Column নির্দেশ করে:

Predicted Class

Confusion Matrix তৈরি করা

Scikit-Learn ব্যবহার করে:

```
C/C++

1conf_mat = confusion_matrix(
```

```
    y_test,
    y_pred
)

conf_mat
```

উদাহরণ Output:

C/C++

```
array([
 [12,  0,  0],
 [ 0, 14,  1],
 [ 0,  0,  9]
])
```

Confusion Matrix ব্যাখ্যা

প্রথম Row:

C/C++

```
[12,  0,  0]
```

Actual Class 0-এর:

- 12টি সঠিকভাবে Class 0 Predict হয়েছে
- কোনো ভুল Prediction হয়নি

দ্বিতীয় Row:

C/C++

```
[ 0, 14,  1]
```

Actual Class 1-এর:

- 14টি সঠিকভাবে Predict হয়েছে
- 1টি ভুল করে Class 2 Predict হয়েছে

তৃতীয় Row:

```
C/C++


1[0, 0, 9]
```

Actual Class 2-এর:

9টি সঠিকভাবে Predict হয়েছে
কোনো ভুল হয়নি

Confusion Matrix Visualize করা

Heatmap ব্যবহার করে Confusion Matrix আরও সহজে দেখা যায়।

```
C/C++


1plt.figure(figsize=(6,4))
2
3sns.heatmap(
4    conf_mat,
5    annot=True,
6    fmt='d'
7)
8
9plt.xlabel("Predicted")
10plt.ylabel("Actual")
11
12plt.show()
```

Precision কী?

Precision নির্দেশ করে:

Model যেগুলোকে Positive বলেছে, তার মধ্যে কতগুলো আসলেই Positive ছিল।

Formula:

Precision = TP / (TP + FP)

Precision উদাহরণ

ধরুন:

TP = 90

FP = 10

তাহলে,

Precision = 90 / (90 + 10)

Precision = 0.90

অর্থাৎ

90%

Precision কখন গুরুত্বপূর্ণ?

যেসব ক্ষেত্রে False Positive কমানো গুরুত্বপূর্ণ:

- Spam Detection
- Fraud Detection
- Medical Diagnosis

সেখানে Precision গুরুত্বপূর্ণ Metric।

Recall কী?

Recall নির্দেশ করে:

Actual Positive-এর মধ্যে কতগুলো Positive সঠিকভাবে শনাক্ত করা হয়েছে।

Formula:

Recall = TP / (TP + FN)

Recall উদাহরণ

ধরি:

TP = 90

FN = 5

তাহলে,

Recall = 90 / (90 + 5)

Recall = 0.947

অর্থাৎ

94.7%

Recall কখন গুরুত্বপূর্ণ?

যেসব ক্ষেত্রে Positive Case Miss করা বিপজ্জনক:

- Cancer Detection
- Disease Detection
- Fraud Detection

সেখানে Recall গুরুত্বপূর্ণ।

F1-Score কী?

F1-Score হলো Precision এবং Recall-এর Harmonic Mean।

Formula:

F1-Score = 2 * (Precision * Recall) / (Precision + Recall)

F1-Score কেন ব্যবহার করা হয়?

কখনো Precision বেশি হতে পারে কিন্তু Recall কম হতে পারে।

আবার Recall বেশি হতে পারে কিন্তু Precision কম হতে পারে।

F1-Score এই দুইটির মধ্যে Balance তৈরি করে।

Classification Report কী?

Classification Report একসাথে দেখায়:

- Precision
- Recall
- F1-Score
- Support

প্রতিটি Class-এর জন্য আলাদাভাবে।

Classification Report তৈরি করা

C/C++

```
print(
    classification_report(
        y_test,
        y_pred
    )
)
```

উদাহরণ Output:

C/C++

```
              precision    recall    f1-score   support

          1       1.00      1.00      1.00        12
          2       1.00      0.93      0.97        15
          3       0.90      1.00      0.95         9
```

```
7accuracy                           0.97      36
8macro avg       0.97      0.98      0.97      36
9weighted avg    0.97      0.97      0.97      36
```

Classification Report-এর Column ব্যাখ্যা

Precision

কতগুলো Predicted Positive আসলে Positive।

Recall

কতগুলো Actual Positive সঠিকভাবে শনাক্ত হয়েছে।

F1-Score

Precision এবং Recall-এর Balanced Score।

Support

প্রতিটি Class-এর Sample সংখ্যা।

Macro Average কী?

Macro Average প্রতিটি Class-এর Metric-এর সাধারণ Average।

Formula:

Macro Average = (Metric1 + Metric2 + ... + MetricN) / N

এখানে সব Class সমান গুরুত্ব পায়।

Weighted Average কী?

Weighted Average প্রতিটি Class-এর Sample সংখ্যার উপর ভিত্তি করে Average গণনা করে।

যে Class-এ বেশি Sample থাকে তার প্রভাব বেশি হয়।

সম্পূর্ণ Evaluation Code

C/C++

```
from sklearn.metrics import (
    accuracy_score,
    confusion_matrix,
    classification_report
)

accuracy = accuracy_score(
    y_test,
    y_pred
)

print("Accuracy:")
print(accuracy)

conf_mat = confusion_matrix(
    y_test,
    y_pred
)

print("\nConfusion Matrix:")
print(conf_mat)

print("\nClassification Report:")
print(
    classification_report(
        y_test,
        y_pred
    )
)
```

Accuracy, Confusion Matrix এবং Classification Report-এর তুলনা

Accuracy
সামগ্রিক Performance দেখায়।

Confusion Matrix
কোন Class কোথায় ভুল হচ্ছে তা দেখায়।

Classification Report
প্রতিটি Class-এর বিস্তারিত Performance দেখায়।

বাস্তব Project-এ কী ব্যবহার করা উচিত?

শুধু Accuracy দেখে Model বিচার করা উচিত নয়।

সাধারণত:

- Accuracy দেখা হয়।
- Confusion Matrix বিশ্লেষণ করা হয়।
- Classification Report দেখা হয়।
- Precision, Recall এবং F1-Score মূল্যায়ন করা হয়।

এভাবে Model-এর প্রকৃত Performance সম্পর্কে পরিষ্কার ধারণা পাওয়া যায়।

সারসংক্ষেপ

Confusion Matrix এবং Classification Report Machine Learning Model Evaluation-এর অত্যন্ত গুরুত্বপূর্ণ অংশ। Accuracy শুধুমাত্র সামগ্রিক সঠিকতার হার দেখায়, কিন্তু কোন Class কোথায় ভুল হচ্ছে তা বোঝায় না। Confusion Matrix সঠিক এবং ভুল Prediction-এর বিস্তারিত চিত্র প্রদান করে, আর Classification Report Precision, Recall, F1-Score এবং Support-এর মাধ্যমে প্রতিটি Class-এর Performance বিশ্লেষণ করে। তাই বাস্তব Project-এ KNN বা যেকোনো Classification Model মূল্যায়নের সময় Accuracy-এর পাশাপাশি Confusion Matrix এবং Classification Report অবশ্যই ব্যবহার করা উচিত।

16.8 Simulating different settings of KNN

ভূমিকা

পূর্ববর্তী মডিউলগুলোতে আমরা KNN-এর মৌলিক ধারণা, Distance Metrics, K Value, Model Implementation এবং Performance Evaluation সম্পর্কে আলোচনা করেছি।

কিন্তু বাস্তব Machine Learning Project-এ একটি গুরুত্বপূর্ণ প্রশ্ন হলো:

"কোন K Value সবচেয়ে ভালো?"

অথবা

"Euclidean Distance ভালো কাজ করবে নাকি Manhattan Distance?"

এই প্রশ্নগুলোর উত্তর পাওয়ার জন্য আমাদের KNN-এর বিভিন্ন Setting পরীক্ষা করতে হয়।

এই প্রক্রিয়াকে বলা যায়:

KNN Simulation অথবা **Hyperparameter Experimentation**

এই মডিউলে আমরা বিভিন্ন K Value এবং Distance Metric ব্যবহার করে Model-এর Performance তুলনা করব।

KNN-এর Performance কেন পরিবর্তিত হয়?

KNN-এর Prediction মূলত নির্ভর করে:

- K-এর মান
- Distance Metric
- Feature Scaling
- Dataset-এর প্রকৃতি

একই Dataset-এ K = 5 এবং K = 25 ব্যবহার করলে ভিন্ন Accuracy পাওয়া স্বাভাবিক।

একইভাবে Manhattan Distance এবং Euclidean Distance ব্যবহার করলেও ফলাফল পরিবর্তিত হতে পারে।

Simulation-এর উদ্দেশ্য

Simulation-এর মাধ্যমে আমরা জানতে পারি:

- কোন K Value সবচেয়ে ভালো
- কোন Distance Metric ভালো কাজ করছে
- Model Overfitting করছে কিনা
- Model Underfitting করছে কিনা

আমাদের Experiment

আমরা দুটি Experiment করব।

Experiment 1

Minkowski Distance

p = 1

অর্থাৎ Manhattan Distance

Experiment 2

Minkowski Distance

p = 2

অর্থাৎ Euclidean Distance

উভয় ক্ষেত্রেই:

K = 5 থেকে K = 29 পর্যন্ত পরিবর্তন করা হবে।

প্রতিটি K-এর জন্য:

- Accuracy
- Precision
- Recall

পরিমাপ করা হবে।

Performance Store করার জন্য List তৈরি

C/C++

```
1performance = []
2acc = []
```

এখানে:

C/C++

```
1performance
```

সকল Metric সংরক্ষণ করবে।

C/C++

```
1acc
```

শুধুমাত্র Accuracy সংরক্ষণ করবে।

Experiment 1: Manhattan Distance (p = 1)

কোড

C/C++

```
1performance = []
2acc = []
3
4for k in range(5,30):
5
6    knn_model = Pipeline(
7        steps=[
8            ('scaler', StandardScaler()),
9            ('model',
10             KNeighborsClassifier(
11                 n_neighbors=k,
12                 metric='minkowski',
13                 p=1
14             ))
15        ]
16    )
17
18    knn_model.fit(X_train, y_train)
19
20    y_pred = knn_model.predict(X_test)
21
22    accuracy = accuracy_score(
23        y_test,
24        y_pred
25    )
26
27    precision = precision_score(
```

```
        y_test,
        y_pred,
        average='weighted'
    )

    recall = recall_score(
        y_test,
        y_pred,
        average='weighted'
    )

    acc.append(accuracy)

    performance.append(
        (
            k,
            (
                accuracy,
                precision,
                recall
            )
        )
    )

print(performance)
```

ব্যাখ্যা

Loop-এর মাধ্যমে K-এর মান পরিবর্তন করা হচ্ছে।

C/C++

```
for k in range(5,30)
```

অর্থাৎ:

K = 5
K = 6
K = 7
...
K = 29
প্রতিবার একটি নতুন KNN Model তৈরি হচ্ছে।

```
C/C++

1KNeighborsClassifier(
2    n_neighbors=k
3)
```

Distance Metric:

```
C/C++

1metric='minkowski'

2p=1
```

এখানে:
p = 1
অর্থাৎ Manhattan Distance।
Model Training:

```
C/C++

1knn_model.fit(
2    X_train,
3    y_train
4)
```

Prediction:

```
C/C++

1y_pred = knn_model.predict(
2    X_test
```

```
3)
```

Performance Metrics:

```
C/C++


1accuracy_score()
2precision_score()
3recall_score()
```

প্রতিটি K-এর ফলাফল Store করা হচ্ছে।

```
C/C++


1performance.append(...)
```

Accuracy Visualization

```
C/C++




1plt.plot(range(5,30), acc)
2
3plt.xlabel("K Value")
4plt.ylabel("Accuracy")
5
6plt.title("K vs Accuracy (Manhattan Distance)")
7
8plt.show()
```

Graph-এর Interpretation

Graph দেখে আমরা বুঝতে পারি:

কোন K-এ Accuracy সবচেয়ে বেশি
Accuracy স্থিতিশীল কিনা
Accuracy দ্রুত কমে যাচ্ছে কিনা

ধরুন Graph থেকে দেখা গেল:

K = 11

বা

K = 13

এর আশেপাশে Accuracy সবচেয়ে বেশি।

তাহলে এগুলো ভালো Candidate হতে পারে।

Experiment 2: Euclidean Distance (p = 2)

এবার একই কাজ Euclidean Distance দিয়ে করা হবে।

কোড

C/C++

```
performance = []
acc = []

for k in range(5,30):

    knn_model = Pipeline(
        steps=[
            ('scaler', StandardScaler()),
            ('model',
              KNeighborsClassifier(
                  n_neighbors=k,
                  metric='minkowski',
                  p=2
              ))
        ]
    )


```

```
    knn_model.fit(X_train, y_train)

    y_pred = knn_model.predict(X_test)

    accuracy = accuracy_score(
        y_test,
        y_pred
    )

    precision = precision_score(
        y_test,
        y_pred,
        average='weighted'
    )

    recall = recall_score(
        y_test,
        y_pred,
        average='weighted'
    )

    acc.append(accuracy)

    performance.append(
        (
            k,
            (
                accuracy,
                precision,
                recall
            )
        )
    )


print(performance)
```

p = 2 এর অর্থ কী?

Minkowski Distance-এর ক্ষেত্রে:

p = 1

→ Manhattan Distance

p = 2

→ Euclidean Distance

তাই নিচের কোড:

```
C/C++

1metric='minkowski'

2p=2
```

বাস্তবে Euclidean Distance ব্যবহার করছে।

Accuracy Plot তৈরি করা

```
C/C++

1plt.plot(range(5,30), acc)
2
3plt.xlabel("K Value")
4plt.ylabel("Accuracy")
5
6plt.title("K vs Accuracy (Euclidean Distance)")
7
8plt.show()
```

দুইটি Distance Metric তুলনা করা

আরও ভালো Visualization-এর জন্য একই Graph-এ দুইটি Curve আঁকা যায়।

C/C++

```
plt.figure(figsize=(8,5))

plt.plot(
    range(5,30),
    acc_manhattan,
    label='Manhattan'
)

plt.plot(
     range(5,30),
     acc_euclidean,
     label='Euclidean'
)

plt.xlabel("K Value")
plt.ylabel("Accuracy")

plt.legend()

plt.show()
```

Simulation থেকে কী শেখা যায়?

এই Experiment থেকে আমরা জানতে পারি:

কোন K Value সবচেয়ে ভালো

সবচেয়ে বেশি Accuracy প্রদানকারী K নির্বাচন করা যায়।

কোন Distance Metric ভালো

Manhattan এবং Euclidean-এর মধ্যে কোনটি ভালো কাজ করছে তা দেখা যায়।

Model Stable কিনা

K পরিবর্তনের সাথে Accuracy খুব বেশি ওঠানামা করলে Model Stable নয়।

Overfitting এবং Underfitting

ছোট K:

সাধারণত Overfitting-এর ঝুঁকি বেশি।

বড় K:

সাধারণত Underfitting-এর ঝুঁকি বেশি।

Overfitting-এর উদাহরণ

ধরুন:

K = 1

Accuracy = 100%

Training Data-এর প্রতি Model অত্যন্ত সংবেদনশীল হয়ে যায়।

Noise-এর কারণেও Prediction পরিবর্তিত হতে পারে।

এটি Overfitting-এর লক্ষণ।

Underfitting-এর উদাহরণ

ধরুন:

K = 50

Accuracy = 75%

এক্ষেত্রে Model অতিরিক্ত Generalization করছে।

ফলে গুরুত্বপূর্ণ Pattern হারিয়ে যাচ্ছে।

এটি Underfitting-এর লক্ষণ।

Best K নির্বাচন করার কৌশল

Graph পর্যবেক্ষণ করে:

- সর্বোচ্চ Accuracy খুঁজুন।
- Accuracy স্থিতিশীল কিনা দেখুন।
- খুব ছোট K এড়িয়ে চলুন।
- খুব বড় K এড়িয়ে চলুন।
- Validation Performance বিবেচনা করুন।

Simulation কেন গুরুত্বপূর্ণ?

বাস্তবে কোনো Dataset-এর জন্য আগে থেকে বলা যায় না:

- K = 5 ভালো হবে
- K = 7 ভালো হবে
- Manhattan Distance ভালো হবে
- Euclidean Distance ভালো হবে

তাই Experiment করে যাচাই করা প্রয়োজন।
এ কারণেই Hyperparameter Tuning Machine Learning-এর একটি গুরুত্বপূর্ণ ধাপ।

পরবর্তী ধাপ: **GridSearchCV**

এই মডিউলে আমরা Manual Simulation করেছি।
অর্থাৎ:

- K পরিবর্তন করেছি
- Metric পরিবর্তন করেছি
- Accuracy তুলনা করেছি

কিন্তু বাস্তব Project-এ এটি স্বয়ংক্রিয়ভাবে করার জন্য Scikit-Learn-এর GridSearchCV ব্যবহার করা হয়।
পরবর্তী মডিউলে আমরা GridSearchCV ব্যবহার করে Best Hyperparameter খুঁজে বের করার পদ্ধতি শিখব।

সারসংক্ষেপ

KNN-এর Performance Distance Metric এবং K Value-এর উপর সরাসরি নির্ভর করে। তাই বিভিন্ন K এবং Distance Metric ব্যবহার করে Simulation করা অত্যন্ত গুরুত্বপূর্ণ। এই মডিউলে আমরা Manhattan Distance (p=1) এবং Euclidean Distance (p=2) ব্যবহার করে K = 5 থেকে K = 29 পর্যন্ত বিভিন্ন Setting পরীক্ষা করেছি এবং Accuracy, Precision ও Recall পর্যবেক্ষণ করেছি। এই ধরনের Simulation আমাদের Best K Value নির্বাচন করতে সাহায্য করে এবং Model-এর Stability, Overfitting ও Underfitting সম্পর্কে গুরুত্বপূর্ণ ধারণা প্রদান করে।

16.9 GridSearchCV Intuition and Implementation

ভূমিকা

পূর্ববর্তী মডিউলে আমরা KNN-এর বিভিন্ন Settings Simulation করে দেখেছি। সেখানে আমরা Manual ভাবে K-এর বিভিন্ন মান এবং Distance Metric পরিবর্তন করে Accuracy পর্যবেক্ষণ করেছি।
কিন্তু বাস্তব Machine Learning Project-এ Hyperparameter-এর সংখ্যা অনেক বেশি হতে পারে। তখন প্রতিটি Combination হাতে হাতে পরীক্ষা করা সময়সাপেক্ষ এবং কষ্টসাধ্য।
এই সমস্যার সমাধানের জন্য Scikit-Learn আমাদের একটি শক্তিশালী Tool প্রদান করে, যার নাম:
GridSearchCV
GridSearchCV স্বয়ংক্রিয়ভাবে বিভিন্ন Hyperparameter Combination পরীক্ষা করে এবং সবচেয়ে ভালো Combination নির্বাচন করে।

Hyperparameter কী?

Machine Learning-এ Hyperparameter হলো এমন Parameter যা Training-এর আগে নির্ধারণ করা হয়।

KNN-এর ক্ষেত্রে Hyperparameter-এর উদাহরণ:

- n_neighbors
- weights
- metric
- p

উদাহরণ:

```
C/C++

1KNeighborsClassifier(
2    n_neighbors=5,
3    weights='uniform',
4    p=2
5)
```

এখানে:

- n_neighbors
- weights
- p

সবগুলোই Hyperparameter।

Grid Search কী?

Grid Search হলো এমন একটি পদ্ধতি যেখানে আমরা সম্ভাব্য Hyperparameter-এর বিভিন্ন মান নির্ধারণ করি এবং Model সবগুলো Combination পরীক্ষা করে।

ধরুন:

n_neighbors:

```
C/C++

1[3, 5, 7]
```

weights:

```
C/C++

1['uniform', 'distance']
```

তাহলে মোট Combination হবে:

```
C/C++

1(3, uniform)
2(3, distance)
3
4(5, uniform)
5(5, distance)
6
7(7, uniform)
8(7, distance)
```

অর্থাৎ:

3 × 2 = 6 Combination

GridSearchCV এই ৬টি Combination-এর প্রতিটিকে পরীক্ষা করবে।

GridSearchCV কী?

GridSearchCV হলো Scikit-Learn-এর একটি Utility যা:

- বিভিন্ন Hyperparameter Combination তৈরি করে।
- প্রতিটি Combination Training করে।
- Cross Validation ব্যবহার করে Evaluation করে।
- Best Parameter নির্বাচন করে।
- Best Model Return করে।

কেন GridSearchCV ব্যবহার করা হয়?

Manual Experiment:

C/C++

```
for k in range(1,30):
    ...
```

এখানে শুধুমাত্র K পরিবর্তন করা হয়েছে।
কিন্তু যদি:

- K-এর 30টি মান
- Weights-এর 2টি মান
- Distance Metric-এর 2টি মান

থাকে, তাহলে:

30 × 2 × 2 = 120

টি Combination পরীক্ষা করতে হবে।

GridSearchCV এই কাজ স্বয়ংক্রিয়ভাবে করে।

প্রয়োজনীয় Library Import

C/C++

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline

from sklearn.datasets import load_wine

from sklearn.neighbors import KNeighborsClassifier

from sklearn.metrics import (
    accuracy_score,
```

```
    precision_score,
    recall_score
)

from sklearn.metrics import (
    confusion_matrix,
    classification_report
)

from sklearn.model_selection import GridSearchCV
```

Dataset Load করা

```
C/C++
```

```
df = load_wine(as_frame=True).frame
```

Feature এবং Target আলাদা করা

```
C/C++
```

```
X = df.drop(['target'], axis=1)

y = df['target']
```

Train-Test Split

C/C++

```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y
)
```

Pipeline তৈরি করা

প্রথমে একটি Basic Pipeline তৈরি করা হবে।

C/C++

```
knn_model = Pipeline(
    steps=[
        ('scaler', StandardScaler()),
        ('knn', KNeighborsClassifier())
    ]
)
```

Pipeline-এর সুবিধা

Pipeline-এর মাধ্যমে:

- Feature Scaling
- Model Training

একসাথে সম্পন্ন হয়।

এতে Data Leakage-এর ঝুঁকি কমে।

Pipeline Fit করা

C/C++

```
knn_model.fit(
    X_train,
    y_train
)
```

Parameter Grid তৈরি করা

Grid Search-এর সবচেয়ে গুরুত্বপূর্ণ অংশ হলো Parameter Grid।

C/C++

```
grid = {
    "knn__n_neighbors": range(1,30),
    "knn__weights": [
        'uniform',
        'distance'
    ],
    "knn__p": [
        1,
        2
    ]
}
```

Parameter Grid ব্যাখ্যা

n_neighbors

C/C++

```
range(1,30)
```

K-এর মান:

1 থেকে 29 পর্যন্ত।

weights

```
C/C++


1['uniform', 'distance']
```

দুই ধরনের Voting Strategy পরীক্ষা করা হবে।

p

```
C/C++


1[1, 2]
```

p = 1

→ Manhattan Distance

p = 2

→ Euclidean Distance

মোট কতগুলো **Combination** তৈরি হবে?

n_neighbors:

29 টি

weights:

2 টি

p:

2 টি

মোট Combination:

29 × 2 × 2

= 116

অর্থাৎ GridSearchCV মোট 116 টি Model পরীক্ষা করবে।

GridSearchCV তৈরি করা

C/C++

```
grid_search = GridSearchCV(
    estimator=knn_model,
    param_grid=grid,
    cv=5
)
```

Parameter ব্যাখ্যা

estimator

C/C++

```
estimator=knn_model
```

যে Model-এর উপর Search চালানো হবে।

param_grid

C/C++

```
param_grid=grid
```

Hyperparameter-এর সম্ভাব্য মান।

cv=5

5-Fold Cross Validation ব্যবহার করা হবে।

Cross Validation কীভাবে কাজ করে?

cv = 5 হলে Training Data-কে ৫ ভাগে ভাগ করা হয়।

ধাপগুলো:

1. প্রথম Fold Validation
2. দ্বিতীয় Fold Validation
3. তৃতীয় Fold Validation
4. চতুর্থ Fold Validation
5. পঞ্চম Fold Validation

প্রতিবার নতুন Fold Validation হিসেবে ব্যবহৃত হয়।
সবশেষে Average Score গণনা করা হয়।

Grid Search Training

C/C++

```
grid_search.fit(
    X_train,
    y_train
)
```

এই ধাপে:

- 116 Combination তৈরি হবে
- প্রতিটি Combination Training হবে
- Cross Validation হবে
- Best Model নির্বাচন হবে

Best Parameter বের করা

C/C++

```
grid_search.best_params_
```

উদাহরণ Output:

C/C++

```
{
    'knn__n_neighbors': 11,
    'knn__p': 2,
    'knn__weights': 'distance'
```

```
5}
```

Output ব্যাখ্যা

Best Model:

```
C/C++

1K = 11
```

Distance:

```
C/C++

1Euclidean Distance
```

Voting Strategy:

```
C/C++

1distance
```

Best Model বের করা

```
C/C++



1grid_search.best_estimator_
```

Output:

```
C/C++

1Pipeline(...)
```

এটি Grid Search দ্বারা নির্বাচিত সর্বোত্তম Model।

Best Model সংরক্ষণ করা

C/C++

```
1model = grid_search.best_estimator_
```

Prediction করা

C/C++

```
1y_pred = model.predict(X_test)
```

Accuracy বের করা

C/C++

```
1accuracy = accuracy_score(
2    y_test,
3    y_pred
4)
5
6print(accuracy)
```

উদাহরণ Output:

C/C++

```
10.9722
```

সম্পূর্ণ কোড

C/C++

```
import numpy as np
import pandas as pd

from sklearn.datasets import load_wine

from sklearn.model_selection import (
    train_test_split,
    GridSearchCV
)

from sklearn.preprocessing import StandardScaler

from sklearn.pipeline import Pipeline

from sklearn.neighbors import KNeighborsClassifier

from sklearn.metrics import accuracy_score

df = load_wine(as_frame=True).frame

X = df.drop(['target'], axis=1)
y = df['target']

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42,
```

```
    stratify=y
)

knn_model = Pipeline(
    steps=[
        ('scaler', StandardScaler()),
        ('knn', KNeighborsClassifier())
    ]
)

grid = {
    "knn__n_neighbors": range(1,30),
    "knn__weights": [
        'uniform',
        'distance'
    ],
    "knn__p": [
        1,
        2
    ]
}

grid_search = GridSearchCV(
    estimator=knn_model,
    param_grid=grid,
    cv=5
)

grid_search.fit(
    X_train,
    y_train
)

print("Best Parameters:")
print(grid_search.best_params_)

```

```
model = grid_search.best_estimator_

y_pred = model.predict(X_test)

accuracy = accuracy_score(
    y_test,
    y_pred
)

print("Accuracy:")
print(accuracy)
```

GridSearchCV-এর সুবিধা

স্বয়ংক্রিয় Hyperparameter Tuning
সব Combination নিজে নিজে পরীক্ষা করে।

Cross Validation ব্যবহার করে
Performance-এর আরও নির্ভরযোগ্য Estimate দেয়।

Best Model নির্বাচন করে
Manual Trial-and-Error-এর প্রয়োজন কমে যায়।

Reproducible Result
প্রতিবার একই Experiment পুনরায় চালানো যায়।

GridSearchCV-এর সীমাবদ্ধতা

Computational Cost বেশি
Hyperparameter বেশি হলে সময় অনেক বেড়ে যায়।

Large Dataset-এ ধীর হতে পারে
Combination যত বেশি হবে Training Time তত বাড়বে।

Memory Usage বেশি হতে পারে
বিশেষ করে বড় Grid-এর ক্ষেত্রে।

বাস্তব Project-এ GridSearchCV

সাধারণ Workflow:

- Dataset প্রস্তুত করা।
- Pipeline তৈরি করা।
- Hyperparameter Grid তৈরি করা।
- GridSearchCV চালানো।
- Best Parameter নির্বাচন করা।
- Test Data-তে Final Evaluation করা।

এটি বর্তমানে Machine Learning Project-এর একটি Standard Practice।

সারসংক্ষেপ

GridSearchCV হলো Scikit-Learn-এর একটি শক্তিশালী Hyperparameter Tuning Tool যা স্বয়ংক্রিয়ভাবে বিভিন্ন Hyperparameter Combination পরীক্ষা করে। KNN-এর ক্ষেত্রে n_neighbors, weights এবং p-এর মতো Hyperparameter-এর বিভিন্ন মান পরীক্ষা করে GridSearchCV সবচেয়ে ভালো Combination নির্বাচন করতে পারে। এটি Cross Validation ব্যবহার করে Model-এর Performance মূল্যায়ন করে এবং Best Estimator Return করে। ফলে Manual Trial-and-Error ছাড়াই একটি Optimized KNN Model তৈরি করা সম্ভব হয়।

16.11 Weights Hyperparameter and Cross Validation

ভূমিকা

পূর্ববর্তী মডিউলে আমরা GridSearchCV ব্যবহার করে KNN-এর Best Hyperparameter খুঁজে বের করেছি। সেখানে আমরা লক্ষ্য করেছি যে KNN-এর একটি গুরুত্বপূর্ণ Hyperparameter হলো:

weights

এছাড়াও GridSearchCV-এর মাধ্যমে আমরা Cross Validation-এর ব্যবহার দেখেছি।

এই মডিউলে আমরা বিস্তারিতভাবে আলোচনা করব:

- Weights Hyperparameter কী
- Uniform এবং Distance Weighting-এর পার্থক্য
- KNN-এ Weighting কেন প্রয়োজন
- Cross Validation কী
- Cross Validation কেন গুরুত্বপূর্ণ
- Scikit-Learn ব্যবহার করে Cross Validation-এর Implementation

Weights Hyperparameter কী?

KNN যখন কোনো নতুন Data Point-এর জন্য Prediction করে, তখন এটি K সংখ্যক Nearest Neighbour নির্বাচন করে।

কিন্তু একটি গুরুত্বপূর্ণ প্রশ্ন হলো:

সব Neighbour কি সমান গুরুত্বপূর্ণ?

উত্তর:

সবসময় নয়।

কারণ কোনো Neighbour খুব কাছাকাছি হতে পারে, আবার কোনো Neighbour তুলনামূলক দূরে হতে পারে।

তাই KNN-এ Weights Hyperparameter ব্যবহার করা হয়।

Weights Hyperparameter-এর ধরন

Scikit-Learn-এ সাধারণত দুটি Option রয়েছে:

```
C/C++

1weights='uniform'
```

এবং

```
C/C++

1weights='distance'
```

Uniform Weighting

এটি KNN-এর Default Behaviour।

এখানে প্রতিটি Neighbour-এর গুরুত্ব সমান।

ধরুন:

K = 5

Nearest Neighbour-এর Class:

```
C/C++

1Red
2Red
```

```
3Blue
4Red
5Blue
```

তাহলে:

Red = 3

Blue = 2

Prediction:

```
C/C++
```

```
1Red
```

এখানে Neighbour কত দূরে আছে তা বিবেচনা করা হয় না।

Distance Weighting

Distance Weighting-এর ক্ষেত্রে কাছের Neighbour বেশি গুরুত্ব পায়।

দূরের Neighbour কম গুরুত্ব পায়।

ধরুন:

Class	Distance
Blue	0.5
Blue	0.6
Red	2.5
Red	2.8
Red	3.0

Uniform Voting হলে:

```
C/C++
```

```
1Blue = 2

2Red = 3
```

Prediction:

```
C/C++

1Red
```

কিন্তু Distance Weighting ব্যবহার করলে:
Blue Point দুটি অনেক কাছাকাছি।
তাই Blue-এর Weight বেশি হবে।
সেক্ষেত্রে Prediction হতে পারে:

```
C/C++

1Blue
```

Distance Weighting-এর ধারণা

সাধারণভাবে Weight Distance-এর বিপরীত আনুপাতিক হয়।
Formula:
Weight = 1 / Distance
অর্থাৎ:
Distance কম
→ Weight বেশি
Distance বেশি
→ Weight কম

Uniform বনাম Distance Weighting

Uniform

সব Neighbour সমান গুরুত্ব পায়।

Distance

কাছের Neighbour বেশি গুরুত্ব পায়।

Uniform-এর সুবিধা

- সহজ
- দ্রুত
- Noise কম থাকলে ভালো কাজ করে

Distance-এর সুবিধা

- Local Pattern ভালো ধরতে পারে

Nearest Point-এর গুরুত্ব বাড়ায়
অনেক Dataset-এ Accuracy উন্নত করে

Scikit-Learn-এ Uniform Weighting

C/C++

```
from sklearn.neighbors import KNeighborsClassifier

knn = KNeighborsClassifier(
    n_neighbors=5,
    weights='uniform'
)
```

Scikit-Learn-এ Distance Weighting

C/C++

```
from sklearn.neighbors import KNeighborsClassifier

knn = KNeighborsClassifier(
    n_neighbors=5,
    weights='distance'
)
```

Uniform এবং Distance Weighting তুলনা

নিচের উদাহরণে দুইটি Model তৈরি করা হয়েছে।

C/C++

```
from sklearn.neighbors import KNeighborsClassifier
```

```
2
3uniform_model = KNeighborsClassifier(
4    n_neighbors=5,
5    weights='uniform'
6)
7
8distance_model = KNeighborsClassifier(
9    n_neighbors=5,
10    weights='distance'
11)
```

Training:

C/C++

```
1uniform_model.fit(
2    X_train,
3    y_train
4)
5
6distance_model.fit(
7    X_train,
8    y_train
9)
```

Prediction:

C/C++

```
1uniform_pred = uniform_model.predict(
2    X_test
3)
4
5distance_pred = distance_model.predict(
6    X_test
)
7
```

Accuracy Comparison:

```
C/C++

from sklearn.metrics import accuracy_score

print(
    accuracy_score(
        y_test,
        uniform_pred
    )
)

print(
    accuracy_score(
        y_test,
        distance_pred
    )
)
```

কোন **Weighting** ব্যবহার করা উচিত?

এর নির্দিষ্ট উত্তর নেই।

এটি Dataset-এর উপর নির্ভর করে।

সাধারণত:

- Uniform দিয়ে শুরু করা যায়
- Distance Weighting-ও পরীক্ষা করা উচিত
- Validation Score দেখে সিদ্ধান্ত নিতে হয়

Cross Validation কী?

Machine Learning-এ একটি সাধারণ সমস্যা হলো:

Model Training Data-তে ভালো কাজ করলেও নতুন Data-তে ভালো নাও করতে পারে।

এই সমস্যা কমানোর জন্য Cross Validation ব্যবহার করা হয়।

Train-Test Split-এর সীমাবদ্ধতা

ধরুন Dataset:
1000 Sample
Train-Test Split:

```
C/C++

1Train = 80%

2Test = 20%
```

এখন যদি Test Dataset খুব সহজ বা খুব কঠিন হয়, তাহলে Evaluation Bias হতে পারে।

Cross Validation-এর ধারণা

Cross Validation Dataset-কে একাধিক ভাগে ভাগ করে।
প্রতিটি ভাগ একবার Validation Dataset হিসেবে ব্যবহৃত হয়।
ফলে Evaluation আরও নির্ভরযোগ্য হয়।

K-Fold Cross Validation

সবচেয়ে জনপ্রিয় Cross Validation Technique হলো:
K-Fold Cross Validation
ধরুন:
K = 5
Dataset-কে 5টি Fold-এ ভাগ করা হলো।

```
C/C++

1Fold 1

2Fold 2

3Fold 3

4Fold 4
```

```
5Fold 5
```

ধাপ ১:

```
C/C++


1Validation = Fold 1
2Training = Fold 2,3,4,5
```

ধাপ ২:

```
C/C++


1Validation = Fold 2
2Training = Fold 1,3,4,5
```

এভাবে প্রতিটি Fold একবার Validation Dataset হিসেবে ব্যবহৃত হয়।

Cross Validation Score

ধরুন ৫টি Fold-এর Accuracy:

```
C/C++


10.94
20.96
30.95
40.97
50.95
```

Average Accuracy:

```
C/C++


1(0.94 + 0.96 + 0.95 + 0.97 + 0.95) / 5
2
3= 0.954
```

এটাই Cross Validation Score।

Cross Validation কেন গুরুত্বপূর্ণ?

More Reliable Evaluation

একটি Test Split-এর উপর নির্ভর করতে হয় না।

Better Generalization Estimate

Model নতুন Data-তে কেমন কাজ করবে তা ভালোভাবে বোঝা যায়।

Hyperparameter Tuning-এ সহায়ক

GridSearchCV মূলত Cross Validation ব্যবহার করে।

Scikit-Learn-এ Cross Validation

প্রথমে প্রয়োজনীয় Module Import করি।

C/C++

```
from sklearn.model_selection import cross_val_score
```

KNN Model তৈরি

C/C++

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier

knn_model = Pipeline(
    steps=[
        ('scaler', StandardScaler()),
        ('knn',
         KNeighborsClassifier(
             n_neighbors=5,
             weights='uniform'
```

```
        ))
    ]
)
```

Cross Validation চালানো

C/C++

```
from sklearn.model_selection import cross_val_score

scores = cross_val_score(
    knn_model,
    X,
    y,
    cv=5
)
```

Scores দেখা

C/C++

```
print(scores)
```

উদাহরণ Output:

C/C++

```
[0.9444 0.9722 0.9444 1.0000 0.9714]
```

Average Score বের করা

C/C++

```
1print(scores.mean())
```

উদাহরণ Output:

C/C++

```
10.9665
```

Cross Validation-এর সম্পূর্ণ উদাহরণ

C/C++

```
1from sklearn.datasets import load_wine
2
3from sklearn.pipeline import Pipeline
4
5from sklearn.preprocessing import StandardScaler
6
7from sklearn.neighbors import KNeighborsClassifier
8
9from sklearn.model_selection import cross_val_score
10
11df = load_wine(as_frame=True).frame
12
13X = df.drop('target', axis=1)
14y = df['target']
15
16knn_model = Pipeline(
```

```
    steps=[
        ('scaler', StandardScaler()),
        ('knn',
         KNeighborsClassifier(
             n_neighbors=5,
             weights='distance'
         ))
    ]
)

scores = cross_val_score(
    knn_model,
    X,
    y,
    cv=5
)

print("Fold Scores:")
print(scores)

print("\nAverage Accuracy:")
print(scores.mean())
```

GridSearchCV এবং Cross Validation-এর সম্পর্ক

অনেক শিক্ষার্থী মনে করে GridSearchCV এবং Cross Validation আলাদা বিষয়।

বাস্তবে:

GridSearchCV-এর ভেতরেই Cross Validation ব্যবহার করা হয়।

উদাহরণ:

```
C/C++
```

```
GridSearchCV(
    estimator=model,
    param_grid=grid,
    cv=5
)
```

এখানে:

```
cv=5
```

মানে প্রতিটি Hyperparameter Combination-এর জন্য 5-Fold Cross Validation চালানো হবে।

বাস্তব **Project-**এ ব্যবহার

সাধারণ Workflow:

- Dataset প্রস্তুত করা।
- Pipeline তৈরি করা।
- Cross Validation ব্যবহার করে Baseline Score বের করা।
- GridSearchCV দিয়ে Hyperparameter Tuning করা।
- Best Model নির্বাচন করা।
- Test Dataset-এ Final Evaluation করা।

সারসংক্ষেপ

KNN-এর Weights Hyperparameter নির্ধারণ করে Neighbour-দের কতটা গুরুত্ব দেওয়া হবে। Uniform Weighting-এ সব Neighbour সমান গুরুত্ব পায়, আর Distance Weighting-এ কাছের Neighbour বেশি গুরুত্ব পায়। কোনটি ভালো কাজ করবে তা Dataset-এর উপর নির্ভর করে। অন্যদিকে Cross Validation Model Evaluation-এর একটি গুরুত্বপূর্ণ পদ্ধতি যা Dataset-কে একাধিক Fold-এ ভাগ করে Model-এর Performance মূল্যায়ন করে। এটি Train-Test Split-এর তুলনায় বেশি নির্ভরযোগ্য ফলাফল প্রদান করে এবং GridSearchCV-এর ভিত্তি হিসেবে কাজ করে। তাই KNN-এর Hyperparameter Tuning এবং Model Evaluation-এর ক্ষেত্রে Weights Hyperparameter এবং Cross Validation উভয়ই অত্যন্ত গুরুত্বপূর্ণ ধারণা।